Smart Logistics Planner

Advanced Algorithms & Programming, mini-project report

Ahmed AlHennawi

1. Introduction

Delivery companies have to solve two problems at the same time, and the two answers pull against each other. If the driver takes parcels in order of urgency, the van zigzags across the city and wastes fuel. If the driver takes the shortest route, urgent parcels such as medicine can sit in the van all day. My tool tries to sit in the middle. Urgency decides what goes on the van, and geography decides the order of the drops.

The version I built works on one specific case. There is one van and one driver, working from one depot, in a city of twenty locations: a warehouse at the centre of a 100 by 100 km grid (node 0) and nineteen delivery points placed at random. The queue holds between 20 and 200 packages, 50 by default, and each package has a destination and an urgency level from 1 (medical or same-hour) to 5 (standard). The van cannot take everything, so it is limited to between 5 and 15 stops per trip, 8 by default. So the tool has to answer two questions: which packages go on this trip, and in what order the van visits them before returning to the depot.

The second question is the Travelling Salesman Problem. With m stops, there are m! possible orders, which gets out of hand quickly: 8 stops give 40,320 orders and 15 stops give more than 1.3 trillion. The first question is a sorting problem, and the sorting key is a small integer with only five possible values. That last detail matters more than it looks, as Section 4 shows.

2. Breaking the problem down

Figure 1 shows how I split the problem before writing any code. It reads from right to left. The goal on the right, optimise last-mile delivery, is too big to program directly, so it breaks into three business goals: get urgent items out first, keep fuel and driving time down, and show the dispatcher what the plan is. Each of those becomes a function with clear inputs and outputs, and the left column lists the actual techniques behind each function. Every item in that left column is a block of code in the notebook.

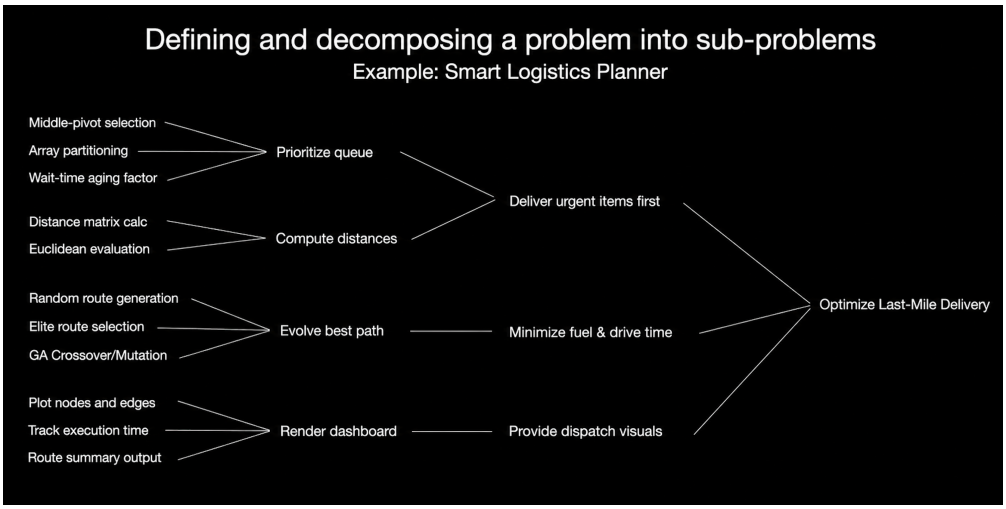


Figure 1: Decomposition of the problem, read right to left, from the main goal down to the techniques that implement it.

Figure 1 says what has to be solved. Figure 2 says in what order things run. There are five stages, and each box lists what it hands to the next stage, so the joins between stages are clear. The sorting stage, for example, takes an unsorted list of packages and returns the same packages in priority order. Nothing after it needs to know how the sorting was done.

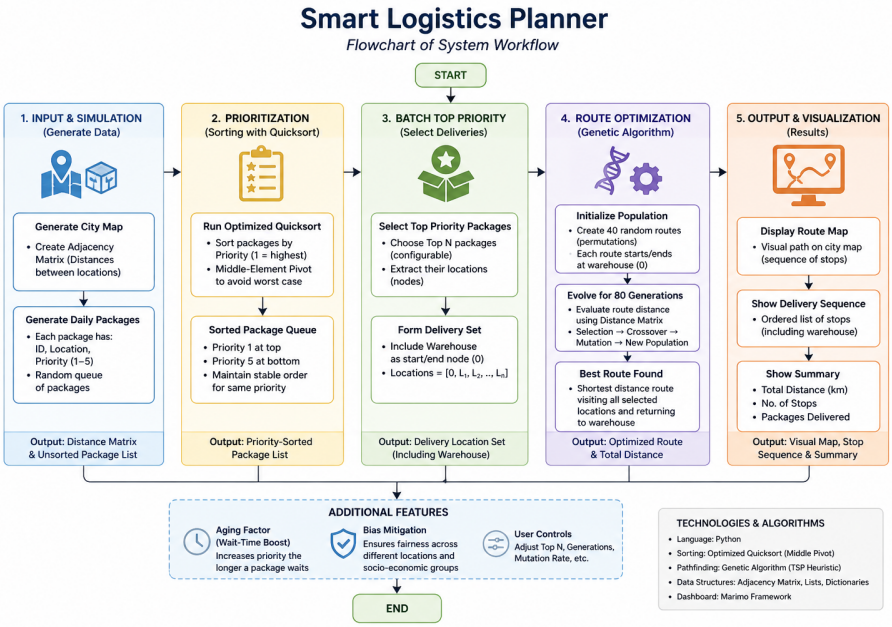


Figure 2: Workflow of the tool, with the five stages, the output of each stage, the technologies used, and the features proposed for later.

3. How the tool works

The tool is a Marimo dashboard with Matplotlib drawing the map. The user moves two sliders, presses Generate City & Queue to create a working day, then presses Optimize Routing to run both algorithms.

Three structures hold everything. The city is kept as a dictionary of coordinates for plotting, and as a 20 by 20 matrix of Euclidean distances, which is what the algorithms actually read. Keeping distances in a matrix means any lookup is one array access instead of a square root. The queue itself is a Python list of dictionaries, and each dictionary holds an ID, a destination node and a priority number. This is the structure the sort receives, so it is worth saying clearly: it is not a list of numbers, it is a list of records, and the sort compares one integer field while the whole record moves with it. Packages arrive in booking order, which I model as random, so the input is genuinely unsorted.

Parameter	Meaning	Value or range
Nodes	Locations in the city, including the depot	20 (node 0 is the depot)
Queue size, n	Packages waiting to be dispatched	20 to 200 (default 50)
Priority	Urgency level on each package	1 (urgent) to 5 (standard)
Truck capacity, m	Stops allowed on one route	5 to 15 (default 8)
Vehicles	Vans and drivers modelled	1, one round trip per run
GA settings	Routes evolved and for how long	40 routes, 80 generations
GA selection	Survival and variation rules	Top 20% kept, swap mutation

Table 1: Parameters the dashboard exposes, and the genetic algorithm settings.

The sorting rule is straightforward. Packages go in ascending order of priority, so all the 1s come before all the 2s, and so on down to 5. Packages with the same priority keep their booking order, because the partitioning I use does not disturb the order inside a group. That means the parcel that has waited longest goes first among equally urgent ones. My Quicksort splits the list into three parts rather than two: the middle element gives the pivot value, every package goes into a lower, equal or higher list, and only the lower and higher lists get sorted again.

Sorting the queue does not decide what goes on the van. That is a separate rule. It walks down the sorted queue, takes the destination of each package, and skips any destination already on the list, since one stop can drop off every parcel going to that address. It stops once the number of stops matches the truck capacity, and whatever is left stays in the queue for the next trip. Urgency picks the stops, and the routing stage then optimises only within that set.

Those stops go to a genetic algorithm. A route is one individual, and its fitness is the length of the full loop out of the depot and back. The algorithm starts with 40 random orders. In each generation, it ranks them by distance, keeps the best 20% untouched, and fills the rest by taking a

parent from the better half and swapping two stops in it. A swap only rearranges stops that are already there, so every route stays valid, and nothing needs repairing. After 80 generations, it returns the shortest route it has, and the dashboard draws it with the stop sequence, the total distance and the time each stage took.

4. Time and space complexity

The sort works on a list of n dictionaries and compares two integers each time, so a comparison costs the same no matter how big the queue is, and counting comparisons is a fair way to judge it. When the pivot splits the list evenly, the work follows $T(n) = 2T(n/2) + \Theta(n)$, which gives $\Theta(n \log n)$. That split is not guaranteed. With a last-element pivot on a list that is already sorted, each partition only peels off one element, the recursion goes n levels deep and the cost falls to $\Theta(n^2)$. A real dispatch queue could easily cause this, since batches from one client often arrive already grouped by service level.

I made two choices to avoid that. The middle-element pivot splits sorted and reverse-sorted input evenly, which are the cases that break the last-element version. To be fair, this makes the slow case unlikely rather than impossible, because a deliberately built input can defeat any fixed pivot rule and only a random pivot avoids that on average. The three-way split matters more here. Since the key only has $k = 5$ values, everything equal to the pivot is finished at that level and never sorted again, so the depth depends on the number of different priorities rather than on n. That gives $\Theta(n \log k)$, which is close to linear for a fixed k of five, and it keeps the sort stable at no extra cost. The cost is memory: the function builds three new lists per call instead of sorting in place, so it needs $\Theta(n)$ extra space. At 200 packages, that is fine. At a million records, I would need an in-place version.

Algorithm	Average time	Worst case	Extra space
Bubble / insertion sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Quicksort (last pivot)	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$
My implementation	$\Theta(n \log k)$, $k = 5$	$\Theta(n^2)$ if defeated	$\Theta(n)$

Table 2: Sorting options for the queue. n is the number of packages, k the number of different priority values.

Big-O describes growth, not speed, so I timed the real thing against a bubble sort on the same records, averaging 20 runs at each size (Figure 3). Going from 100 to 200 packages roughly quadruples the bubble sort time but only doubles mine. At 1,000 packages, it is 0.16 ms against 35.2 ms. At the default 50 packages, both finish in well under a millisecond, so the choice makes no difference to the user today and only starts to matter at the volume a real depot handles.

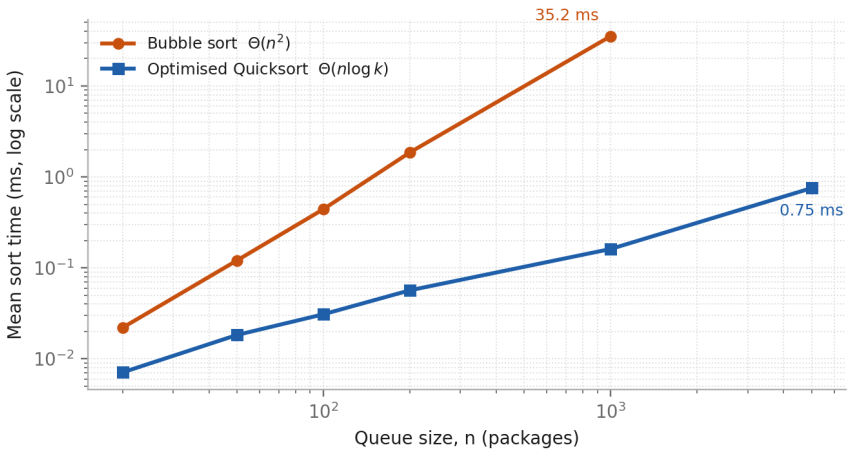


Figure 3: Sorting time against queue size, both axes logarithmic. The steeper line is quadratic growth.

Between the two algorithms, the stop-picking step costs Θ(n · m), since each package is checked against a list of at most m stops. With m capped at 15, that is linear in practice, and a set would make it Θ(n) if the cap ever went up.

Routing is the expensive part. Solving it exactly means checking every order, and the factorial grows too fast to use. Checking all orders for 8 stops took 42 ms in my test. At that rate, 12 stops would take about eight and a half minutes, and 15 stops about sixteen days. The genetic algorithm replaces that with a fixed amount of work: each generation does one distance evaluation per route at Θ(m), plus Θ(P log P) to rank them, then rebuilds the population at Θ(P · m). Over G generations, that is Θ(G · P · (m + log P)), roughly 25,000 additions per run, whatever the instance. The important part is that it grows with m, not with m!, and the timings agree: 7.6 ms for 5 stops and 9.2 ms for 15.

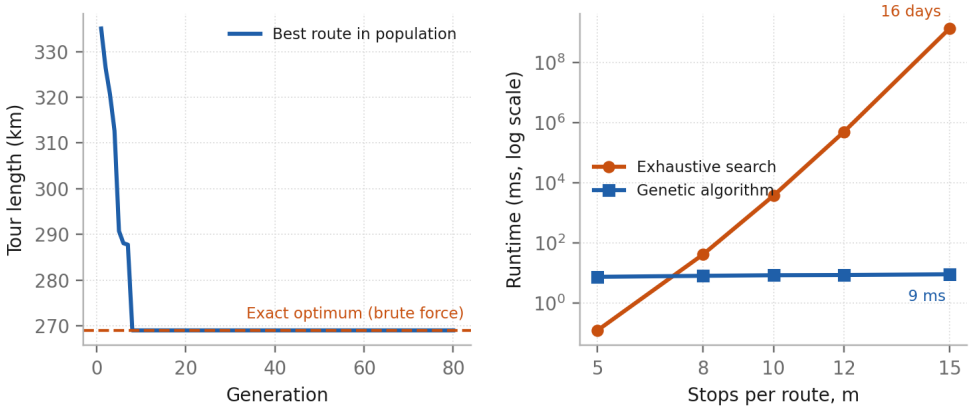


Figure 4: Left, the algorithm converging on an eight-stop route. Right, runtime against number of stops. Exhaustive values past 8 stops are estimated from the measured 8-stop rate.

A heuristic gives up the guarantee of the best answer, so I checked how good the answers actually are. I generated twelve eight-stop problems in different random cities and compared my

route against the true shortest one found by checking every order. It matched the best route all twelve times, and ran about five times faster than the exhaustive method at that size. I would not read too much into it. Eight stops is small, and this is a test result rather than a proof. On bigger or more clustered problems, swapping stops on its own would probably get stuck, and adding a crossover operator or a 2-opt step would be the next thing to try.

Putting it together, one press of Optimise Routing costs $\Theta(n \log k)$ to sort, $\Theta(n \cdot m)$ to pick the stops and $\Theta(G \cdot P \cdot m)$ to route, and the routing term dominates at every setting the sliders allow. For memory, the distance matrix is the biggest item at $\Theta(V^2)$, which is 400 distances for a twenty-node city. The whole thing runs in under 50 ms at the default settings, which is why the dashboard responds instantly.

5. Critical aspects and bias

The weakest point of this design is not speed or accuracy. It is fairness. The van is filled straight from the top of the priority queue, and priority levels are bought, since premium delivery costs more than standard. That creates a socio-economic bias with a geographic pattern: wealthier areas, where more people pay for premium, get served earlier, while addresses on standard shipping pile up at the bottom of the queue. The real problem is that it repeats. Every premium order booked tomorrow jumps ahead of today's leftover standard parcels, so a package can keep getting pushed back. Nothing in the algorithm notices that a package has been waiting, and the metric I optimise, total distance, looks just as good whether someone has waited an hour or a week.

The fix I propose is to stop treating priority as fixed and let waiting time count too, so the sorting key becomes $p_{eff} = p_{tier} - w \cdot h$, where h is hours spent in the queue, and w is a weight the operator sets. If one full level is gained every twelve hours, a standard parcel booked yesterday morning beats a premium parcel booked this morning. That breaks the loop while still giving premium customers the advantage they paid for on the same day. It also fits the code as it is, because the key stays a single number and only the moment it is calculated changes. I should be clear that this is a proposal shown in Figure 2 as future work, not something implemented in the submitted code, where priority stays static. It would also cost the sort its small-k advantage, since ageing makes the keys all different and the sort would go back to ordinary $\Theta(n \log n)$.

A few other simplifications are worth naming, since they limit what I can claim.

- Distances are straight lines. Real cities have road networks, one-way streets and traffic, so real distances are not symmetric and change during the day. Swapping in a matrix from a routing service would fix this without touching either algorithm.
- There is one van making one trip. More vans would make this the Vehicle Routing Problem, where packages have to be shared out between vehicles first.

- Capacity counts stops, not weight or volume, and there are no delivery time slots. Also, the genetic algorithm is random, so two runs on the same input can give different routes.

6. Conclusion

Splitting the problem into two let me pick a suitable algorithm for each half. Quicksort with a three-way split handles urgency in close to linear time here, because there are only five priority values, and it keeps equally urgent parcels in the order they arrived. The genetic algorithm handles the geography with a fixed budget of work instead of a factorial search, and at the sizes this tool supports, it found the shortest route in every test. The limits are just as clear. The sorting argument depends on the five-value key that my own ageing proposal would remove; the routing has no guarantee of being optimal at longer routes, and the tool measures distance, which says nothing about the fairness problem in Section 5. Noticing that was the most useful part of the project for me.